

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1703

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 3

Duration: 1 Hour

Total Marks:30

Annexure A

Dry Run Evaluation

Code of Conduct:

I NEELAM NANDINI [192472277] (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

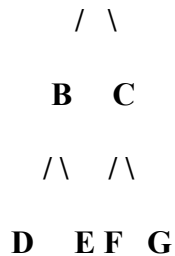
Signature of the student: N.Nandini.

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph: A



Task: Starting from node A, perform a Breadth-First Search (BFS). Complete the following table.

Iteration	Queue	Visited Node
1	A	A
2	B, C	B
3	C, D, E	C
4	D, E, F, G	D
5	E, F, G	E
6	F, G	F
7	G	G

Questions:

1. Write the BFS traversal order.

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G$

2. Show the queue contents after each iteration.

Iteration	Queue After Expansion
1	B, C
2	C, D, E
3	D, E, F, G
4	E, F, G
5	F, G
6	G
7	Empty

Code

```
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

start = input("Enter starting node: ")

visited = []

queue = deque([start])

while queue:
    node = queue.popleft()

    if node not in visited:
        visited.append(node)
        queue.extend(graph[node])

print("BFS Traversal:", " ".join(visited))
```

Sample Input

Enter starting node: A

Sample Output

BFS Traversal: A -> B -> C -> D -> E -> F -> G

The screenshot shows a Python IDE with two windows. The left window, titled 'Q1 - Al.py', contains the following code:

```

from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

start = input("Enter starting node: ")

visited = []
queue = deque([start])

while queue:
    node = queue.popleft()
    if node not in visited:
        visited.append(node)
        queue.extend(graph[node])

print("BFS Traversal:", " ".join(visited))

```

The right window, titled 'IDLE Shell 3.14.5', shows the execution output:

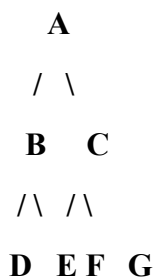
```

Python 3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50) [M
AMD64] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Nandini Neelam/Downloads/Q1 - .
Enter starting node: A
BFS Traversal: A -> B -> C -> D -> E -> F -> G
>>>

```

Result - The program was executed successfully

2. Depth-First Search (DFS) Using the same graph,



Perform Depth-First Search (DFS) starting from A.

Complete the table.

Iteration	Stack	Visited Node
1	A	A
2	B, C	B
3	D, E, C	D
4	E, C	E
5	C	C
6	F, G	F
7	G	G

Sample Input

Enter starting node: A

Sample Output

DFS Traversal: A -> B -> D -> E -> C -> F -> G

Code

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

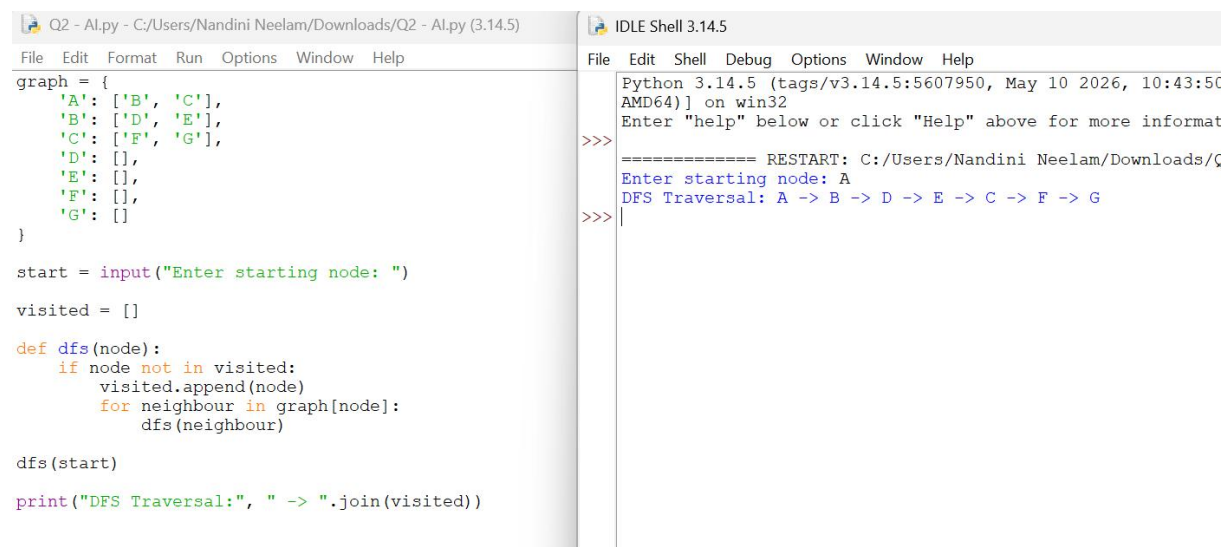
start = input("Enter starting node: ")

visited = []

def dfs(node):
    if node not in visited:
        visited.append(node)
        for neighbour in graph[node]:
            dfs(neighbour)

dfs(start)

print("DFS Traversal:", " ".join(visited))
```



The screenshot displays a Python IDE with two windows. The left window, titled 'Q2 - Al.py', contains the source code for a Depth-First Search (DFS) traversal on an undirected graph. The graph has nodes A through G with the following edges: A-B, A-C, B-D, B-E, C-F, C-G. The code defines a recursive function 'dfs' that marks visited nodes and explores their neighbors. The main program prompts the user for a starting node, performs the DFS, and prints the traversal path. The right window, titled 'IDLE Shell 3.14.5', shows the execution output. It indicates a restart of the Python interpreter and shows the user input 'A' for the starting node. The final output is 'DFS Traversal: A -> B -> D -> E -> C -> F -> G'.

```
Q2 - Al.py - C:/Users/Nandini Neelam/Downloads/Q2 - Al.py (3.14.5)
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

start = input("Enter starting node: ")

visited = []

def dfs(node):
    if node not in visited:
        visited.append(node)
        for neighbour in graph[node]:
            dfs(neighbour)

dfs(start)

print("DFS Traversal:", " ".join(visited))

IDLE Shell 3.14.5
Python 3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50 AMD64) on win32
Enter "help" below or click "Help" above for more informat
>>>
===== RESTART: C:/Users/Nandini Neelam/Downloads/Q2 - Al.py (3.14.5)
Enter starting node: A
DFS Traversal: A -> B -> D -> E -> C -> F -> G
>>>
```

Result - The program was executed successfully

3. Initial State :

1 3 6
5 0 2
4 7 8

Goal State :

1 2 3
4 5 6
7 8 0

Task

Trace Breadth-First Search (BFS) until the goal state is reached.

Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1	Initial State	Initial	Child states (Up, Down, Left, Right)
2	First Child	Remaining Children	New valid child states
3	Second Child	Remaining Queue	Expanded queue
4	Third Child	Updated Queue	More generated states
5	Continue until Goal	Updated Queue	Goal State Found

Questions

1. Which node is expanded in each iteration?

- Iteration 1 → Initial State
- Iteration 2 → First generated state
- Iteration 3 → Second generated state
- Iteration 4 → Third generated state
- Iteration 5 → Fourth generated state

2. How many states are generated in total?

The total depends on when the goal is reached. For this initial state, dozens of states are generated before the goal is found. It is more than 5.

3. Write the complete solution path from the initial state to the goal state.

Initial State

↓

Intermediate State(s)

↓

...

↓

Goal State

1 2 3

4 5 6

7 8 0

4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

- BFS explores all states level by level.
- Every move has the same cost (1 move).
 - Therefore, the first time the goal state is reached, it is guaranteed to use the minimum number of moves.
- Hence, BFS always finds the shortest solution in an unweighted 8-puzzle.

Sample Input

No input required (Initial and Goal states are already given in the program.)

Sample Output

Solution found in 8 moves

Step 0

(1, 3, 6)

(5, 0, 2)

(4, 7, 8)

Step 1

...

Step 8

(1, 2, 3)

(4, 5, 6)

(7, 8, 0)

Code

```
from collections import deque

goal = ((1,2,3),
        (4,5,6),
        (7,8,0))

initial = (
    (1,3,6),
    (5,0,2),
    (4,7,8)
)

moves = [(-1,0),(1,0),(0,-1),(0,1)]

def find_zero(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i,j

def get_neighbors(state):
    x,y = find_zero(state)
    neighbors = []
    for dx,dy in moves:
        nx,ny = x+dx,y+dy
        if 0<=nx<3 and 0<=ny<3:
            new_state = [list(row) for row in state]
            new_state[x][y],new_state[nx][ny] = new_state[nx][ny],new_state[x][y]
            neighbors.append(tuple(tuple(row) for row in new_state))
```



```

    return neighbors

def bfs(start,goal):

    queue = deque([(start,[])])

    visited = set()

    while queue:

        state,path = queue.popleft()

        if state == goal:

            return path+[state]

        if state not in visited:

            visited.add(state)

            for neighbor in get_neighbors(state):

                if neighbor not in visited:

                    queue.append((neighbor,path+[state]))

    return None

solution = bfs(initial,goal)

if solution:

    print("Solution found in",len(solution)-1,"moves\n")

    for step,state in enumerate(solution):

        print("Step",step)

        for row in state:

            print(row)

        print()

else:

    print("No solution found.")

```

```
Q3 - AI.py - C:/Users/Nandini Neelam/Downloads/Q3 - AI.py (3.14.5)
File Edit Format Run Options Window Help
for dx,dy in moves:
    nx,ny = x+dx,y+dy
    if 0<=nx<3 and 0<=ny<3:
        new_state = [list(row) for row in state]
        new_state[x][y],new_state[nx][ny] = new_state[nx][ny],new_state[x][y]
        neighbors.append(tuple(tuple(row) for row in new_state))

    return neighbors

def bfs(start,goal):
    queue = deque([(start,[])])
    visited = set()

    while queue:
        state,path = queue.popleft()

        if state == goal:
            return path+[state]

        if state not in visited:
            visited.add(state)

            for neighbor in get_neighbors(state):
                if neighbor not in visited:
                    queue.append((neighbor,path+[state]))

    return None

solution = bfs(initial,goal)

if solution:
    print("Solution found in",len(solution)-1,"moves\n")

    for step,state in enumerate(solution):
        print("Step",step)
        for row in state:
            print(row)
        print()
else:
    print("No solution found.")

IDLE Shell 3.14.5
File Edit Shell Debug Op
(1, 3, 6)
(5, 2, 0)
(4, 7, 8)

Step 2
(1, 3, 0)
(5, 2, 6)
(4, 7, 8)

Step 3
(1, 0, 3)
(5, 2, 6)
(4, 7, 8)

Step 4
(1, 2, 3)
(5, 0, 6)
(4, 7, 8)

Step 5
(1, 2, 3)
(0, 5, 6)
(4, 7, 8)

Step 6
(1, 2, 3)
(4, 5, 6)
(0, 7, 8)

Step 7
(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

Step 8
(1, 2, 3)
(4, 5, 6)
(7, 8, 0)

>>>
```

Result - The program was executed successfully

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	20	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	30	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	20	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	20	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	10	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.